Course End Project - Sales Analysis

- As a part of this, the very first step is that we are going to import the necessary libraries, and they are as follows:

```
[1]  import numpy as np
     import pandas as pd
     from sklearn.preprocessing import MinMaxScaler
     import matplotlib.pyplot as plt
     import seaborn as sns
     import datetime
```

- In the next, we are going to read the .csv file and store that as a dataframe called ```df```.

```
[2]  df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Sales.csv")
```

- Now, print the dataframe df and as certain the number of rows and colums of data.

```
df.head()
```

```
df.tail()
```

- For performing the data normalization, first of all, we need to separate the numerical and non numerica data. We will create a new dataframe called df_dataonly from the existing df object, as follows:

```
df_dataonly = df[['Unit','Sales']]
```

- Now, from the MinMaxScalar object, create a normalize object

```
[8]  normalize = MinMaxScaler()
```

- Next, invoke the fit_transform() method, and pass this newly created object called df_dataonly. Let's name the resulting object as normalize_data.

```
[9]  normalize_data = normalize.fit_transform(df_dataonly)
```

- As you can see, normalize_data object is a ndarray of 2 columns and 7560 rows. The first columns in the Unit, and the second column is the Sales data. The normalization will render normalize the data for each column between 0 and 1. You can test it as follows:
- normalize_data[:, [0]] will list all the values of normailzed Unit values, while normalize_data[:, [1]] will list all the values of normalized Sales values.

```
[10] normalize_data[:,[0]]

array([[0.0952381 ],
       [0.0952381 ],
       [0.03174603],
       ...,
       [0.20634921],
       [0.14285714],
       [0.17460317]])
```

```
normalize_data[:,[1]]

array([[0.0952381 ],
       [0.0952381 ],
       [0.03174603],
       ...,
       [0.20634921],
       [0.14285714],
       [0.17460317]])
```

- Now, check the min and max values of each of the column. Min should be 0.0 and max should be 1.0, for Unit as well as Sales column. Let's print them as follows:

```
print(normalize_data[:,[0]].min(),normalize_data[:,[0]].max())

0.0 0.9999999999999999
```

```
[13] print(normalize_data[:,[1]].min(),normalize_data[:,[1]].max())

0.0 1.0
```

- Let's plot the Date versus Unit and Date versus Sales line plot for the entire season. Note the values of Unit and Sales are summed up for each day.

```
dates = df['Date']
df_Unit_and_Sales = df.groupby(by = 'Date', axis ='index').sum(numeric_only=True)
```

```
df_Unit_and_Sales
```

Date	Unit	Sales
2020-12-01	1786	4465000
2020-11-01	1208	3020000
2020-10-01	1488	3720000
2020-12-02	1826	4565000
2020-11-02	1090	2725000
...	...	...
2020-11-29	1294	3235000
2020-10-29	1529	3822500
2020-12-30	1836	4590000
2020-11-30	1214	3035000
2020-10-30	1521	3802500

90 rows × 2 columns

- **Now, let's chunk this quarterly data into monthly data and perform the analysis. For each of the month, let's get the sub-dataframe, using the loc feature of dataframe, and pass the range by date. For example, for the month of October, the range for loc will be '2020-10-01':'2020-10-30', and we will be capruting it as df_dec. Similarly, for the month of November and December, we have sub-dataframe objects df_nov and df.dec. These are shown in the next 3 steps.**

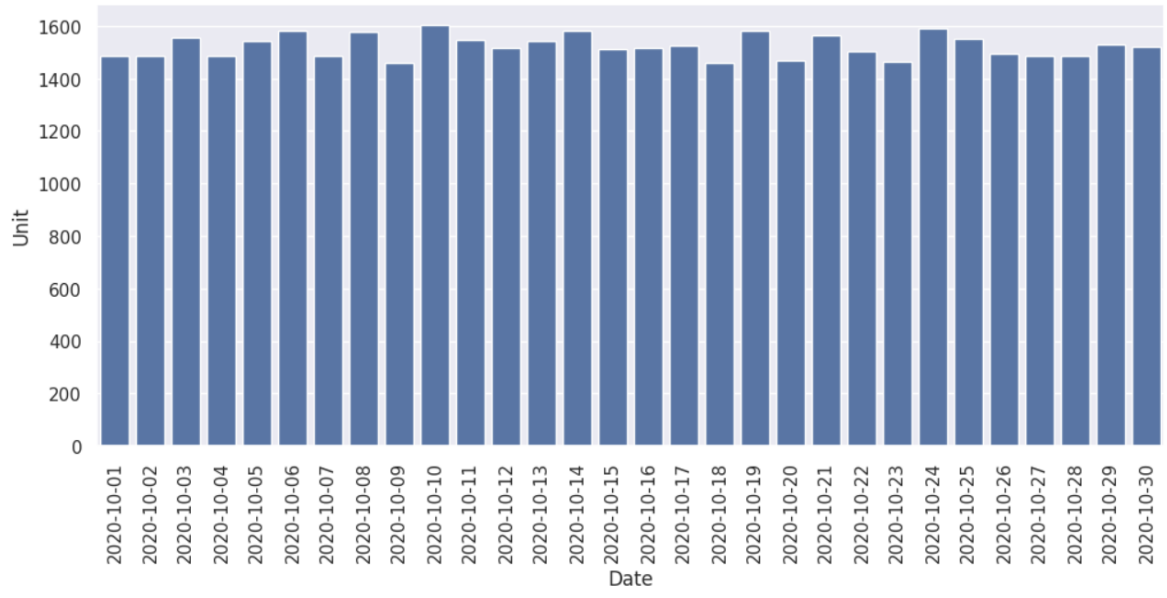
```
[57] df_Unit_and_Sales.index = pd.to_datetime(df_Unit_and_Sales.index)
     df_oct = df_Unit_and_Sales[df_Unit_and_Sales.index.month == 10]
     df_nov = df_Unit_and_Sales[df_Unit_and_Sales.index.month == 11]
     df_dec = df_Unit_and_Sales[df_Unit_and_Sales.index.month == 12]
```

```
[58] df_oct
```

```
df_nov
```

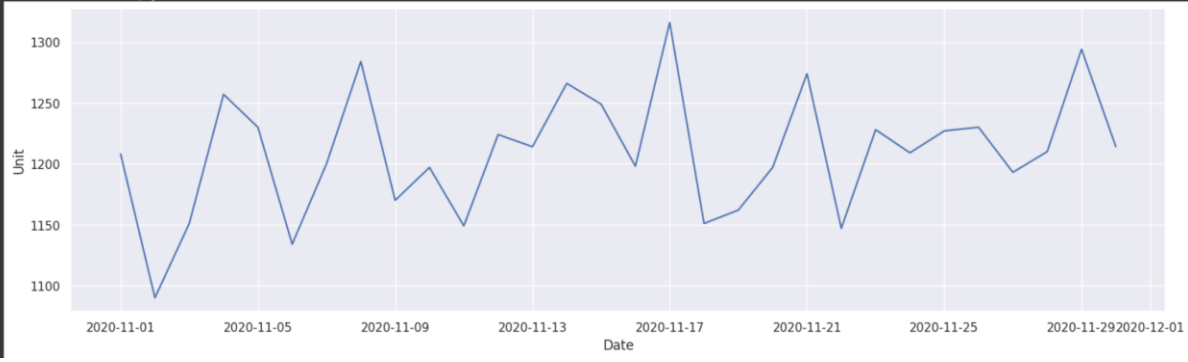
```
df_dec
```

```
df_oct.index
plt.figure(figsize=(12,5))
sns.barplot(x = df_oct.index, y = 'Unit', data = df_oct)
plt.xticks(rotation=90)
plt.show()
```

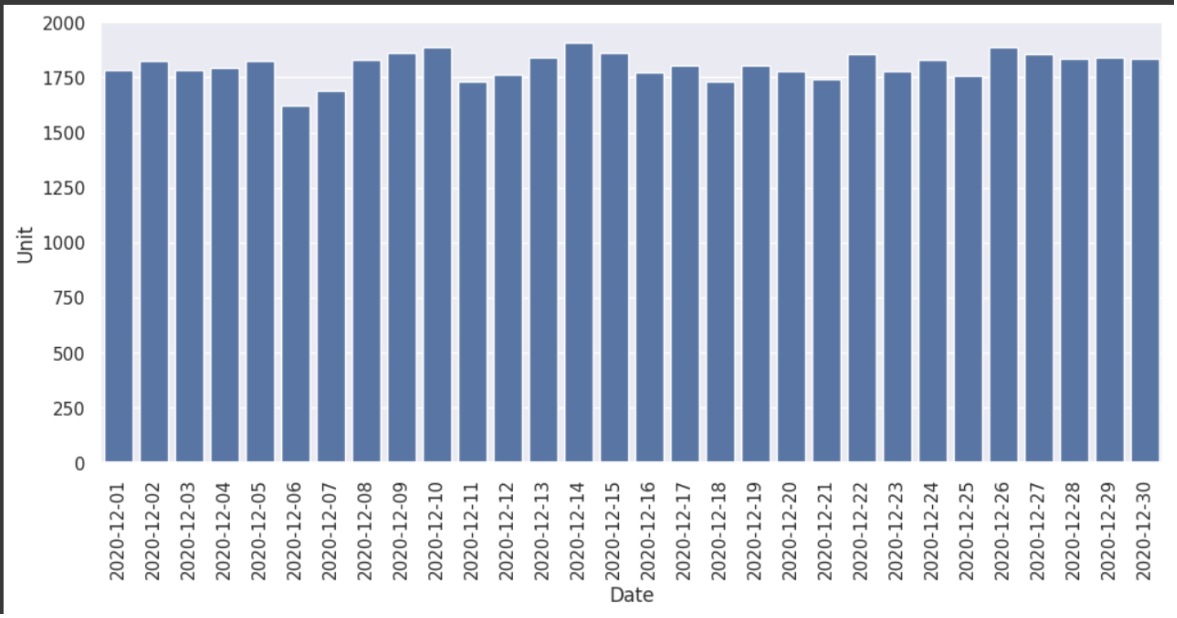



```
plt.figure(figsize=(18,5))
sns.lineplot(x = df_nov.index, y ='Unit', data = df_nov)
```

```
<Axes: xlabel='Date', ylabel='Unit'>
```



```
df_dec.index
plt.figure(figsize=(12,5))
sns.barplot(x = df_dec.index, y = 'Unit', data=df_dec)
plt.xticks(rotation=90)
plt.show()
```



Data Description

- **Describing the data will give you the very first level information on the data with the basic information on the data such as count, mean, std (standard deviation), min, max and the quartiles. You will use the describe() command on the dataframe to get it. All the values are for the entire three month period.**

```
df.describe()
```

	Unit	Sales
count	7560.000000	7560.000000
mean	18.005423	45013.558201
std	12.901403	32253.506944
min	2.000000	5000.000000
25%	8.000000	20000.000000
50%	14.000000	35000.000000
75%	26.000000	65000.000000
max	65.000000	162500.000000

```
df_oct.describe()
```

	Unit	Sales
count	30.000000	3.000000e+01
mean	1523.866667	3.809667e+06
std	43.041867	1.076047e+05
min	1458.000000	3.645000e+06
25%	1487.250000	3.718125e+06
50%	1519.500000	3.798750e+06
75%	1554.500000	3.886250e+06
max	1602.000000	4.005000e+06

```
df_nov.describe()
```

	Unit	Sales
count	30.000000	3.000000e+01
mean	1209.100000	3.022750e+06
std	51.177413	1.279435e+05
min	1090.000000	2.725000e+06
25%	1175.750000	2.939375e+06
50%	1209.500000	3.023750e+06
75%	1230.000000	3.075000e+06
max	1316.000000	3.290000e+06

```
[67] df_dec.describe()
```

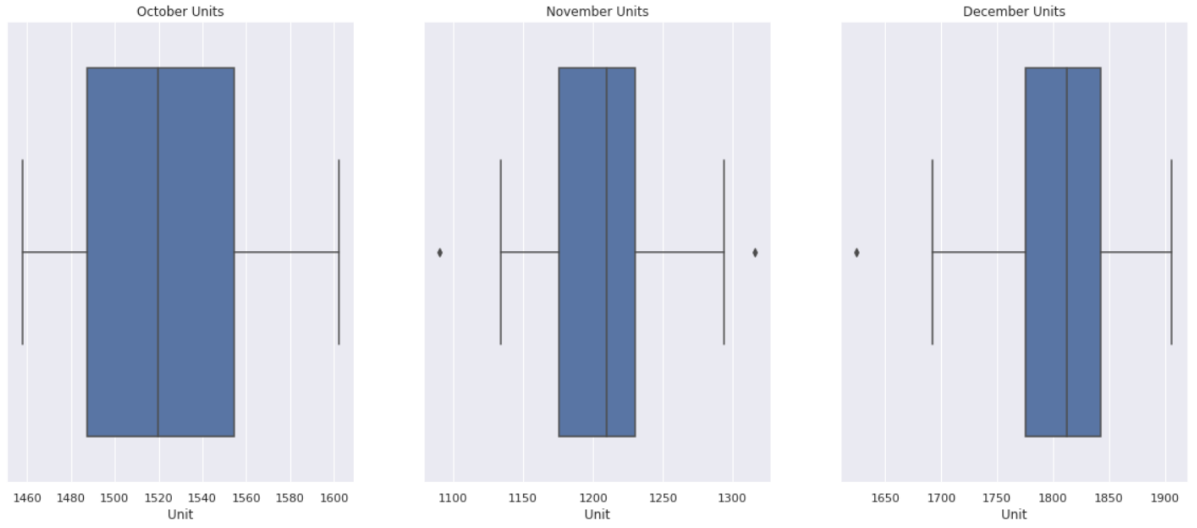
	Unit	Sales
count	30.000000	3.000000e+01
mean	1804.400000	4.511000e+06
std	61.370329	1.534258e+05
min	1624.000000	4.060000e+06
25%	1775.750000	4.439375e+06
50%	1812.500000	4.531250e+06
75%	1842.750000	4.606875e+06
max	1906.000000	4.765000e+06

Box Plot Analysis

Unit Analysis

```
sns.set(rc={'figure.figsize':(20,8)})
fig, axes = plt.subplots(1, 3)
bp_oct = sns.boxplot(x='Unit', data=df_oct, ax=axes[0])
bp_nov = sns.boxplot(x='Unit', data=df_nov, ax=axes[1])
bp_dec = sns.boxplot(x='Unit', data=df_dec, ax=axes[2])
bp_oct.set(title='October Units')
bp_nov.set(title='November Units')
bp_dec.set(title='December Units')
```

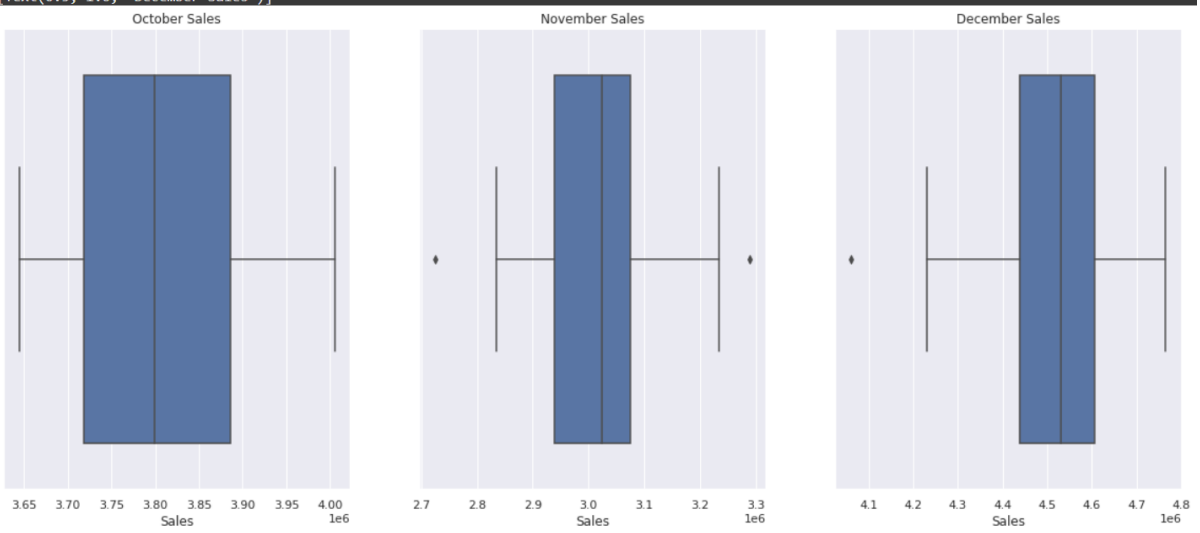
```
[Text(0.5, 1.0, 'December Units')]
```



Sales Analysis

```
sns.set(rc={'figure.figsize':(20,8)})
fig, axes = plt.subplots(1, 3)
bp_oct = sns.boxplot(x='Sales', data=df_oct, ax=axes[0])
bp_nov = sns.boxplot(x='Sales', data=df_nov, ax=axes[1])
bp_dec = sns.boxplot(x='Sales', data=df_dec, ax=axes[2])
bp_oct.set(title='October Sales')
bp_nov.set(title='November Sales')
bp_dec.set(title='December Sales')
```

```
[Text(0.5, 1.0, 'December Sales')]
```



Monthly Plots and Analysis

In the above section, we separated the data monthly wise and performed the top-level description to get the main statistics of the sales. In this section, we will plot, month-wise and do a comparative study on the numbers.

Overall Unit and Sales figures

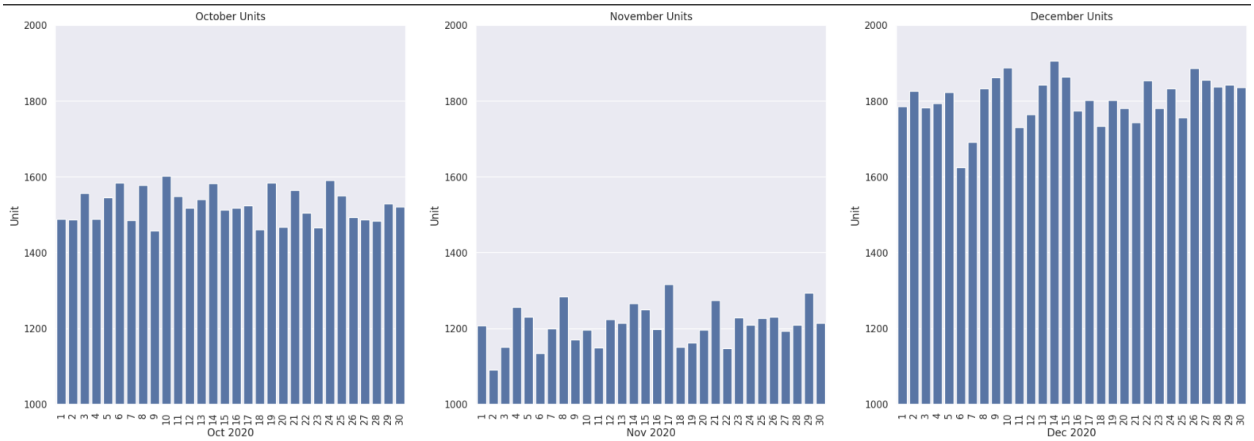
```
[29] oct_days = df_oct.index.day
     oct_days.astype('str')
     nov_days = df_nov.index.day
     nov_days.astype('str')
     dec_days = df_dec.index.day
     dec_days.astype('str')
```

```
Index(['1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13',
       '14', '15', '16', '17', '18', '19', '20', '21', '22', '23', '24', '25',
       '26', '27', '28', '29', '30'],
      dtype='object', name='Date')
```

Units sold in October, November and December

- **For this, we can use bar plot of seaborn, and sub-plots feature of Matplotlib. All the three months are plotted on the same level. As you can see:**

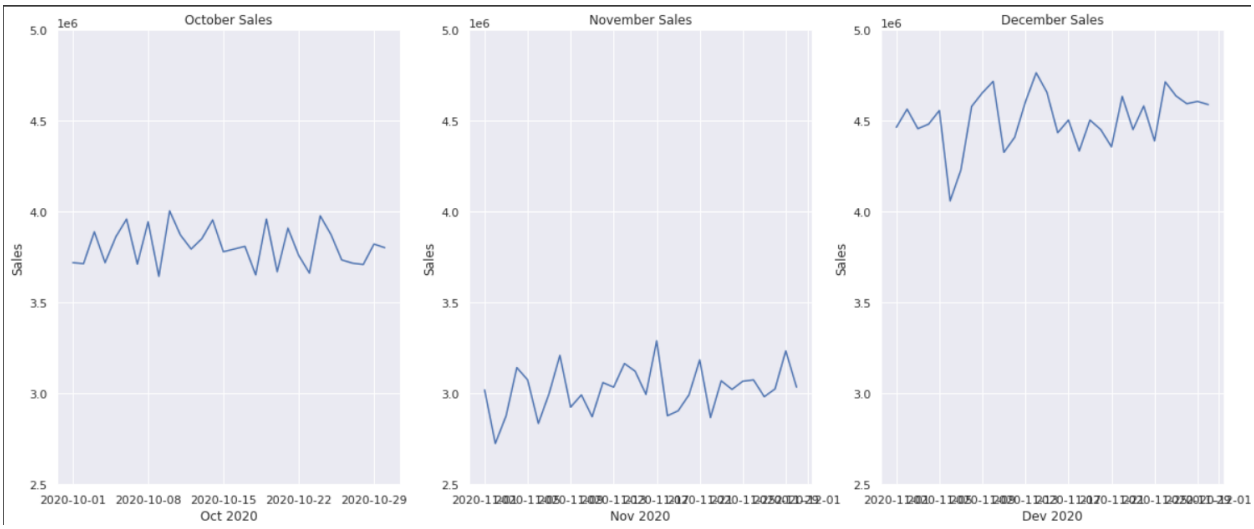
```
sns.set(rc={'figure.figsize':(25,8)})
fig, axes = plt.subplots(1,3)
bp_oct = sns.barplot(x =df_oct.index, y = 'Unit', data=df_oct, ax=axes[0])
bp_nov = sns.barplot(x =df_nov.index, y = 'Unit', data=df_nov, ax=axes[1])
bp_dec = sns.barplot(x =df_dec.index, y = 'Unit', data=df_dec, ax=axes[2])
bp_oct.set(title= 'October Units', xlabel= 'Oct 2020')
bp_nov.set(title= 'November Units', xlabel= 'Nov 2020')
bp_dec.set(title= 'December Units', xlabel= 'Dec 2020')
bp_oct.set(ylim=(1000, 2000))
bp_nov.set(ylim=(1000, 2000))
bp_dec.set(ylim=(1000, 2000))
o = bp_oct.set_xticklabels(oct_days, rotation=90)
n = bp_nov.set_xticklabels(nov_days, rotation=90)
d = bp_dec.set_xticklabels(dec_days, rotation=90)
```



Sales numbers for October, November and December

- **We are using lineplot from seaborn, and once again, we use sub-plot features of Matplotlib, and the sales figures are plotted for each month and plotted one-next-to-the other. Also, you can see, the y axis is same for all the plots, and based on the available numbers, December was most productive in terms of the monthly sales.**

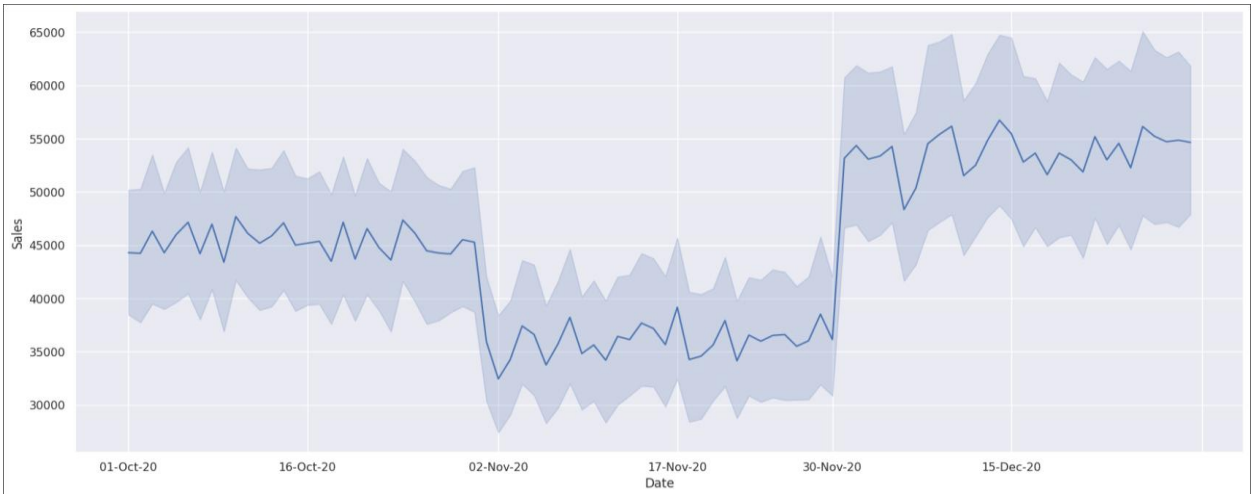
```
[31] # sns.lineplot(x = 'Date', y ='Unit', data = df)
     import matplotlib
     sns.set(rc={'figure.figsize' : (30,8)})
     fig, axes = plt.subplots(1,3)
     lp_oct = sns.lineplot(x = df_oct.index, y = 'Sales', data = df_oct, ax = axes[0])
     lp_nov = sns.lineplot(x = df_nov.index, y = 'Sales', data = df_nov, ax = axes[1])
     lp_dec = sns.lineplot(x = df_dec.index, y = 'Sales', data = df_dec, ax = axes[2])
     lp_oct.set(ylim=(2.5e6, 5.0e6))
     lp_nov.set(ylim=(2.5e6, 5.0e6))
     lp_dec.set(ylim=(2.5e6, 5.0e6))
     lp_oct.set(xlabel ='Oct 2020', title = 'October Sales')
     lp_nov.set(xlabel ='Nov 2020', title = 'November Sales')
     lp_dec.set(xlabel ='Dec 2020', title = 'December Sales')
     df_oct.index = pd.to_datetime(df_oct.index)
     df_nov.index = pd.to_datetime(df_nov.index)
     df_dec.index = pd.to_datetime(df_dec.index)
     loc = matplotlib.dates.DayLocator(bymonthday=range(1,30,7))
     lp_oct.xaxis.set_major_locator(loc)
     lp_nov.xaxis.set_major_locator(loc)
     lp_dec.xaxis.set_major_locator(loc)
```



Consolidated 3 month Sales plot

- **The following is the typical Sales figures, using the lineplot of seaborn. Notice that we are using the original dataframe object df, and this has all the three months of data, as against the previous three, where the three sub-dataframes are separated by groupby() feature of pandas. One more point to be noted here. This plot is different from the plot above, and this is evident from the values of y-axis. The difference here is that the above figures are consolidated data for each month, including the Group and Time data, which are categorical in nature.**

```
[32] sns.set(rc={'figure.figsize':(20,8)})
     c_s = sns.lineplot(x = 'Date', y = 'Sales', data = df)
     loc = matplotlib.dates.DayLocator(bymonthday=range(1, 30, 15))
     c_s.xaxis.set_major_locator(loc)
```

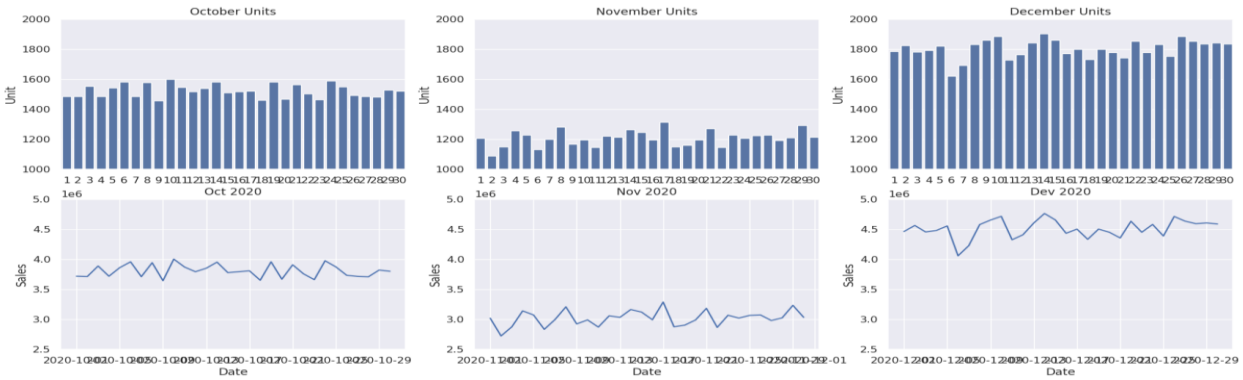


- **One more aspect of this is that this line plot is inclusive of stastical estimation and errors bars that are superimposed on the line plot. This is the special feature of seaborn. This is provided eventhough it is not asked for.**

Comprehensive Snapshot

- **The following are the complete month-wise snapshots that shows monthly Units sales on the top-row, and monthly Sales in the bottom. The sub-plot feature is used to get a complets comprehensive snapshot for the three months.**

```
fig, axes = plt.subplots(2, 3)

bp_oct = sns.barplot(x = df_oct.index, y='Unit', data=df_oct, ax=axes[0,0])
bp_nov = sns.barplot(x = df_nov.index, y='Unit', data=df_nov, ax=axes[0,1])
bp_dec = sns.barplot(x = df_dec.index, y='Unit', data=df_dec, ax=axes[0,2])

bp_oct.set(xlabel='Oct 2020', title='October Units')
bp_nov.set(xlabel='Nov 2020', title='November Units')
bp_dec.set(xlabel='Dev 2020', title='December Units')

bp_oct.set(ylim=(1000, 2000))
bp_nov.set(ylim=(1000, 2000))
bp_dec.set(ylim=(1000, 2000))

o = bp_oct.set_xticklabels(oct_days)
n = bp_nov.set_xticklabels(nov_days)
d = bp_dec.set_xticklabels(dec_days)

lp_oct = sns.lineplot(x = df_oct.index, y='Sales', data=df_oct, ax=axes[1,0])
lp_nov = sns.lineplot(x = df_nov.index, y='Sales', data=df_nov, ax=axes[1,1])
lp_dec = sns.lineplot(x = df_dec.index, y='Sales', data=df_dec, ax=axes[1,2])

lp_oct.set(ylim=(2.5e6, 5.0e6))
lp_nov.set(ylim=(2.5e6, 5.0e6))
lp_dec.set(ylim=(2.5e6, 5.0e6))
```



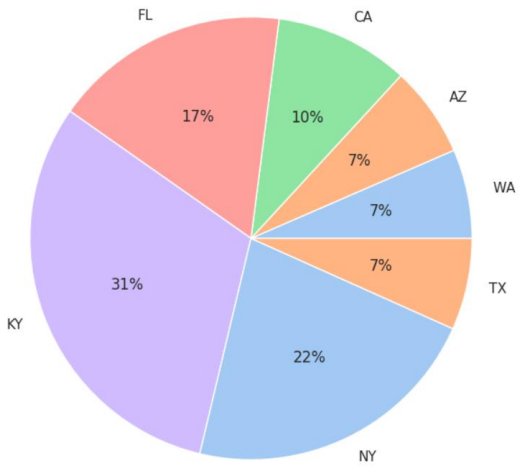
Analysis of Statewise sales

- **In the next, we are going to perform the analysis based on the categorical data of the problem. There are three main categories - State, Group and Time. In the first, we are going to pivot our main dataframe df indexed on State, and also we are providing 2 aggfunctions for computing - sum and mean. The reporting will be state-wise, and we are calling it as state_pivot. As you can see from the output, both sum and mean for Unit and Sales are reported.**

```
[34] df_numeric = df[['Unit','Sales', 'State']]
     state_pivot = pd.pivot_table(df_numeric, index = 'State', aggfunc = ['mean', 'sum'])
     state_pivot
```

	mean		sum	
	Sales	Unit	Sales	Unit
State				
WA	20511.574074	8.204630	22152500	8861
AZ	20907.407407	8.362963	22580000	9032
CA	30942.129630	12.376852	33417500	13367
FL	54497.685185	21.799074	58857500	23543
KY	97745.370370	39.098148	105565000	42226
NY	69416.666667	27.766667	74970000	29988
TX	21074.074074	8.429630	22760000	9104

```
labels = state_pivot['mean']['Sales'].index.to_list()
# print(labels)
colors = sns.color_palette('pastel')[0:5]
plt.pie(state_pivot['mean']['Sales'], labels=labels, colors=colors, autopct='%.0f%%')
plt.show()
```



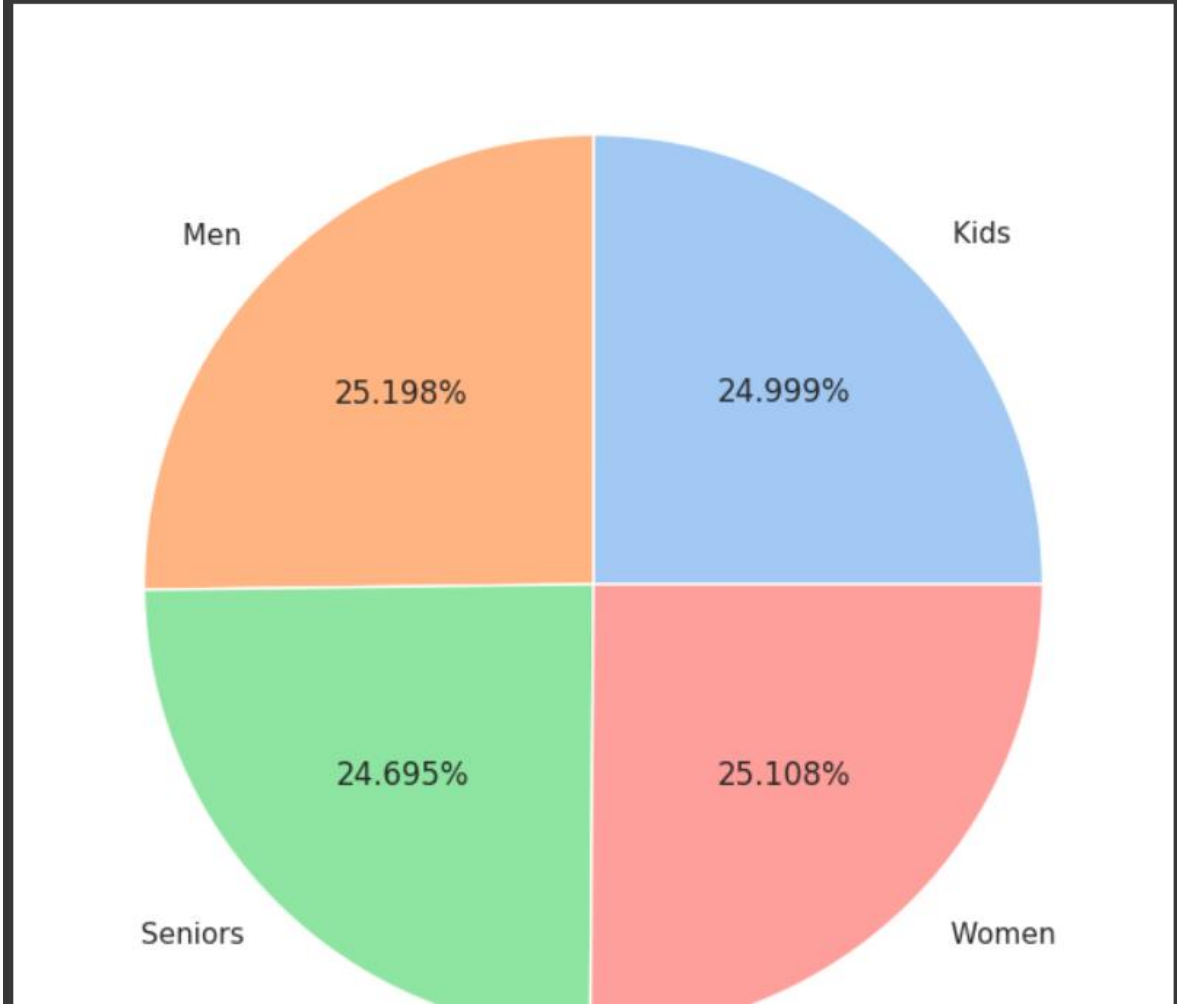
Groupwise Analysis

- **In the next, we are going to perform the analysis based on the next categorical data of the problem - Group. Here, we are going to pivot our main dataframe df indexed on Group, and also we are providing the same two aggfunctions for computing - sum and mean. The reporting will be state-wise, and we are calling it as group_pivot. As you can see from the output, both sum and mean for Unit and Sales are reported.**

```
group_pivot = pd.pivot_table(df, index='Group', values=['Unit', 'Sales'], aggfunc=['sum', 'mean'])
group_pivot
```

	sum		mean	
	Sales	Unit	Sales	Unit
Group				
Kids	85072500	34029	45011.904762	18.004762
Men	85750000	34300	45370.370370	18.148148
Seniors	84037500	33615	44464.285714	17.785714
Women	85442500	34177	45207.671958	18.083069

```
labels = group_pivot['mean']['Sales'].index.to_list()
# print(labels)
colors = sns.color_palette('pastel')[0:5]
plt.pie(group_pivot['mean']['Sales'], labels=labels, colors=colors, autopct='%.3f%%')
plt.show()
```



- **The group-wise distribution presents us an interesting picture. Men, Women, Kids and Seniors have equal share, in terms of the value of sales for the entire three-month period.**

Timewise Analysis

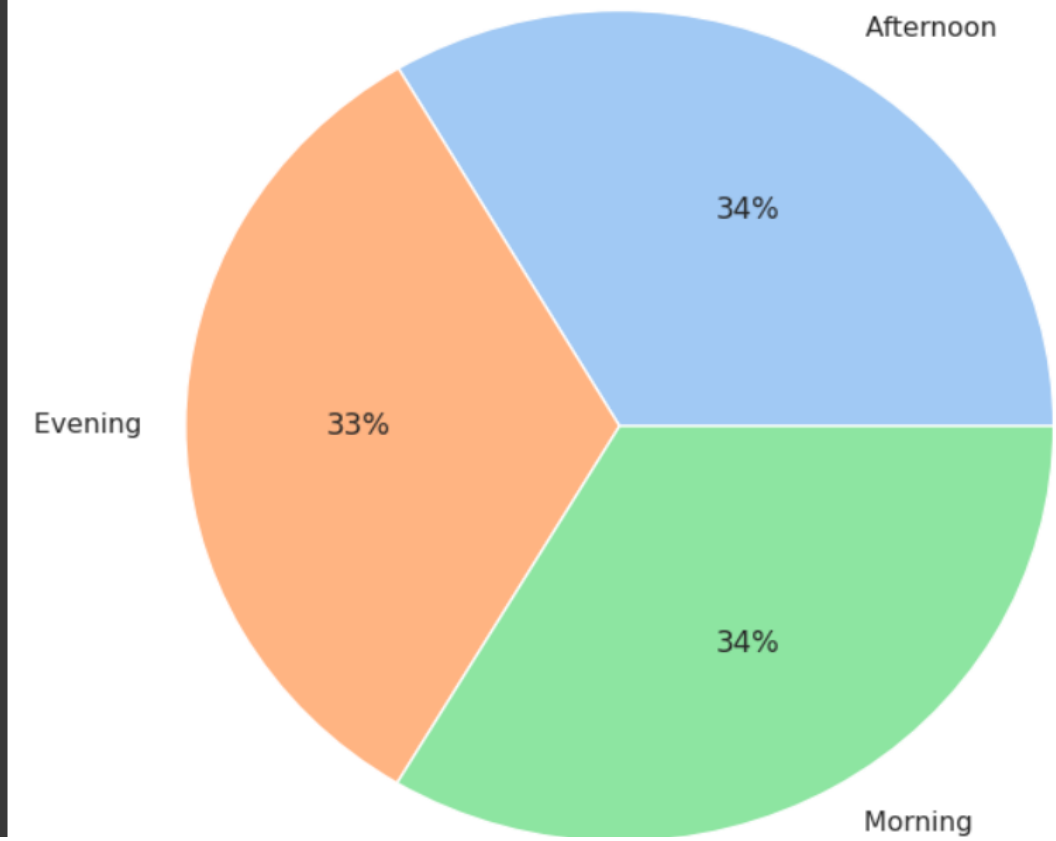
- **In the next, we are going to perform the analysis based on the next categorical data of the problem - Time. Here, we are going to pivot our main dataframe df indexed on Time, and also we are providing the same two aggfunctions for computing**

- sum and mean. The reporting will be state-wise, and we are calling it as time_pivot. As you can see from the output, both sum and mean for Unit and Sales are reported.

```
time_pivot = pd.pivot_table(df, index='Time', values=['Unit', 'Sales'], aggfunc=['sum', 'mean'])
time_pivot
```

	sum		mean	
	Sales	Unit	Sales	Unit
Time				
Afternoon	114007500	45603	45241.071429	18.096429
Evening	112087500	44835	44479.166667	17.791667
Morning	114207500	45683	45320.436508	18.128175

```
labels = time_pivot['mean']['Sales'].index.to_list()
# print(labels)
colors = sns.color_palette('pastel')[0:5]
plt.pie(time_pivot['mean']['Sales'], labels=labels, colors=colors, autopct='%.0f%%')
plt.show()
```



- **The time of sale doesn't matter for AAL, because, almost all the business are equi-distributed in terms of Morning, Afternoon or Evening Sales.**

Report

This report is based on the analysis of three month data - October, November and December 2020. One small detail is that the 31-Oct-2020 is missing in its entirity, and has been ignored. Based on the analysis of monthly data, the apparal business saw a subdued activity in the month of November, and both Unit and Sales saw a muted business. However, the month of December had a stellar performance of approximately 1.5 times the month of November.

In terms of the state-wise analysis, just 3 states contributed to the overall three-month sales - VIC, NSW and SA. The remaining states controbution was just around 30%. The topper-state was VIC, while the laggard state were WA, NT and TAS (with a contribution of just 7% each).

One thing that is clear from the overall numbers is that the apparals were equally popular among different age groups - kids, men, women or seniors.

No particular time was a bad time for AAL for business, as the business was equally busy during all the times of store business hours.